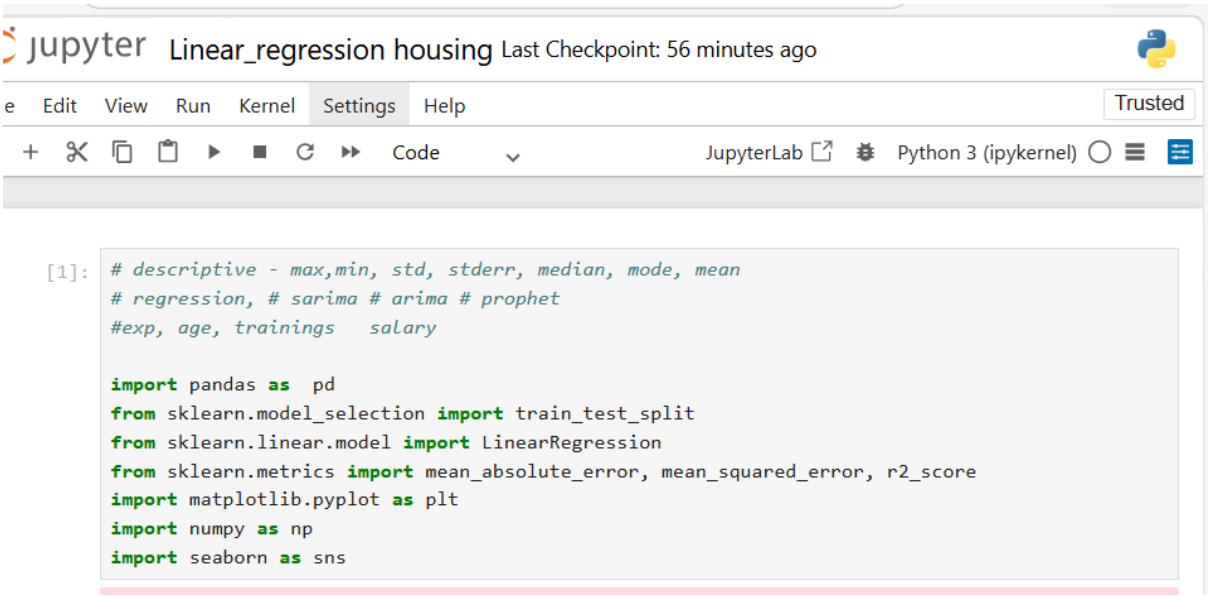
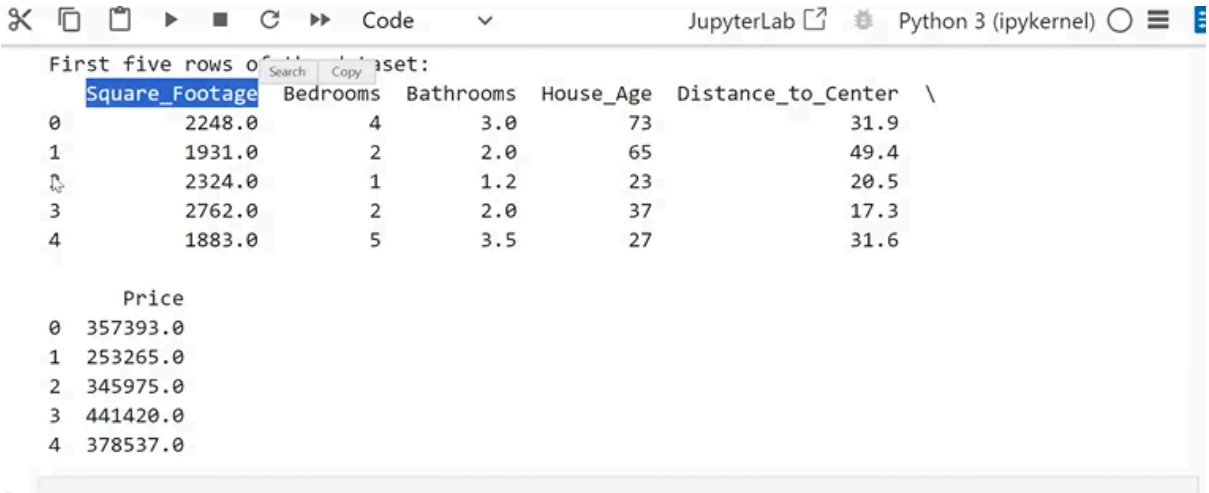
Metrics to evaluate the regression model

Mean_absolute_ error it means+- the amount of the salary

R2_score means if 1 its perfect if 0.8 its great if 0.5 its good below 0.5 its poor




```
# step 2 : Loading the dataset
df = pd.read_csv(r'c:C:\Users\Lenovo-User\Downloads\housing_data.csv')
# Display first 5 rows of the DataFrame to get a quick look
print('First five rows of the dataset:')
print(df.head())
```



```
First five rows of the dataset:
   Square_Footage  Bedrooms  Bathrooms  House_Age  Distance_to_Center  \
0          2248.0         4        3.0         73                31.9
1          1931.0         2        2.0         65                49.4
2          2324.0         1        1.2         23                20.5
3          2762.0         2        2.0         37                17.3
4          1883.0         5        3.5         27                31.6

      Price
0  357393.0
1  253265.0
2  345975.0
3  441420.0
4  378537.0
```

First five rows of the dataset:

	Square_Footage	Bedrooms	Bathrooms	House_Age	Distance_to_Center	Price
0	2248.0	4	3.0	73	31.9	357393.0
1	1931.0	2	2.0	65	49.4	253265.0
2	2324.0	1	1.2	23	20.5	345975.0
3	2762.0	2	2.0	37	17.3	441420.0
4	1883.0	5	3.5	27	31.6	378537.0

```
# Display basic statistical summary of the dataset
print('Statistical Summary of the Dataset:')
df.describe()
```

Statistical Summary of the Dataset:

	Square_Footage	Bedrooms	Bathrooms	House_Age	Distance_to_Center	Price
count	3000.000000	3000.000000	3000.000000	3000.000000	3000.000000	3000.000000
mean	2016.045667	3.001667	2.589233	49.131667	25.454333	348076.390333
std	493.270041	1.409607	0.976117	28.607676	14.313722	87643.877579
min	500.000000	1.000000	1.000000	0.000000	1.000000	25450.000000
25%	1686.500000	2.000000	1.700000	24.000000	12.875000	292014.000000
50%	2012.000000	3.000000	2.600000	49.000000	25.000000	346059.500000
75%	2337.000000	4.000000	3.500000	74.000000	38.300000	407242.500000

```
# Displaying information about the dataframe, including data types and non-null values
print('Information about the dataset:')
df.info()
```

```
Information about the dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 6 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Square_Footage     3000 non-null   float64
 1   Bedrooms           3000 non-null   int64
 2   Bathrooms          3000 non-null   float64
 3   House_Age          3000 non-null   int64
 4   Distance_to_Center 3000 non-null   float64
 5   Price              3000 non-null   float64
dtypes: float64(4), int64(2)
memory usage: 140.8 KB
```

```
from matplotlib import pyplot as plt
# EDA (Visualisations)
# correlation heatmap
# SET SIZE FOR BETTER READABILITY OF THE HEATMAP
plt.figure(figsize =(10,6))
sns.heatmap(df.corr(), annot=True, cmap ='coolwarm' , fmt ='.2f')
plt.savefig('correlation_heatmap.png')
plt.close()
```

```
# Price vs. Square_footage scatter plot
plt.figure(figsize = (10,6))
sns.scatterplot(data=df, x ='Square_Footage', y ='Price' alpha = 0.5)
plt.title(Price vs Square Footage)
#save as a picture(png)
plt.savefig('price_vs_sqft.png')
```


```
# scatter plot(Price vs Distance_to_Center)
import matplotlib.pyplot as plt
#Example data (replace with your dataset)
price = df['Price']
distance = df['Distance_to_center']
plt.figure(figsize=(8,6))
plt.scatter(distance, price, alpha=0.6)
plt.title('Price vs Distance to Center')
plt.xlabel('Distance to Center')
plt.ylabel('Price')
plt.show()
```

```python
# 4. Build Linear Regression Model
# x will contain all column except price (The drop method removes the price column)
#axis = 1 specifies that we are dropping a column and not row
x = df.drop('Price', axis = 1)
y = df['Price']
print('First five rows of features (x)')
x.head()
print('First five rows of target (y):')
y.head()
```

```
First five rows of features (X):
First five rows of target (y):
0    357393.0
1    253265.0
2    345975.0
3    441420.0
4    378537.0
Name: Price, dtype: float64
```

```python
# Training and Testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state =42)
#print the shape of the resulting sets to confirm the split
print(f'Shape of the x_train:{x_train.shape}')
print(f'Shape of x_test: {x_test.Shape}')
print(f'Shape of y_train: {y_train.Shape}')
print(f'Shape of y_test: {y_test.Shape}')
```

```
Shape of the X_train: (2400, 5)
Shape of X_test: (600, 5)
Shape of y_train: (2400,)
Shape of y_test: (600,)
```